Text Embeddings Demystified: From Representation to Clusters

What is an Embedding?

Embedding = **Numeric representation of objects** (words, sentences, documents) in **high-dimensional space**.

- Converts text → numbers so **machines can understand**
- Preserves **semantic meaning**
- Similar objects → **close vectors**

Example:

- “King” → `[0.21, -0.34, 0.85, ...]`
- “Queen” → `[0.19, -0.32, 0.82, ...]`
- Distance between “King” & “Queen” is **small**

What is Vector Space?

Vector Space = a mathematical space where **vectors represent objects**

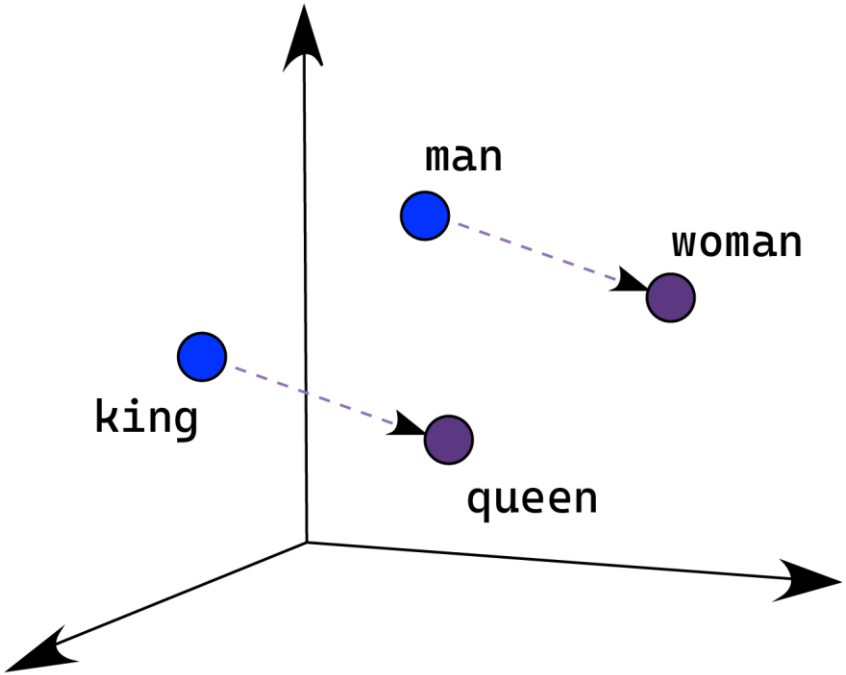
- Each **point/vector** = a word, sentence, or document
- Each **dimension** = a feature of the object
- **Distance & direction** = semantic relationships

Intuition

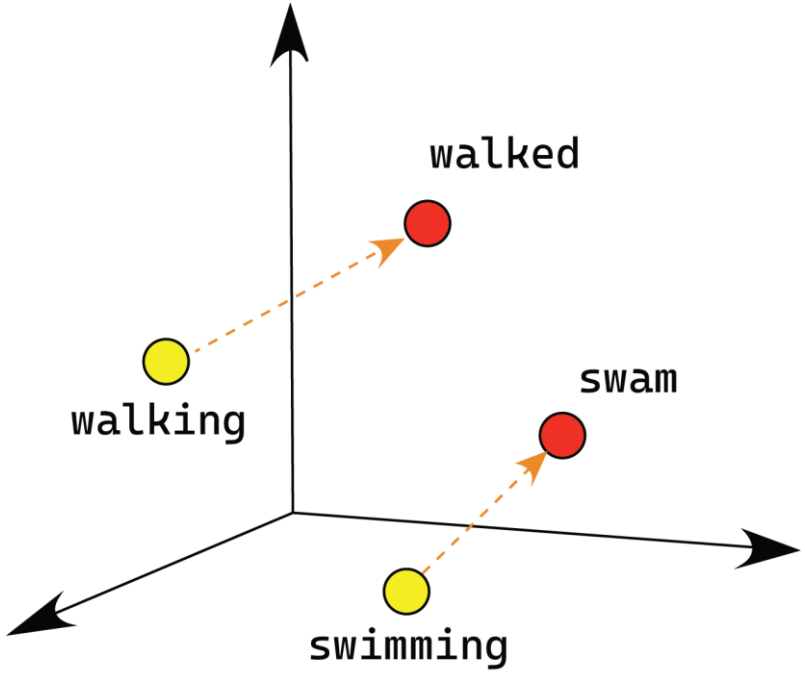
- Words with **similar meaning** → vectors are close
- Words with **different meaning** → vectors are far apart

*Vector spaces allow **quantitative analysis of semantics**.*

What is Vector Space?

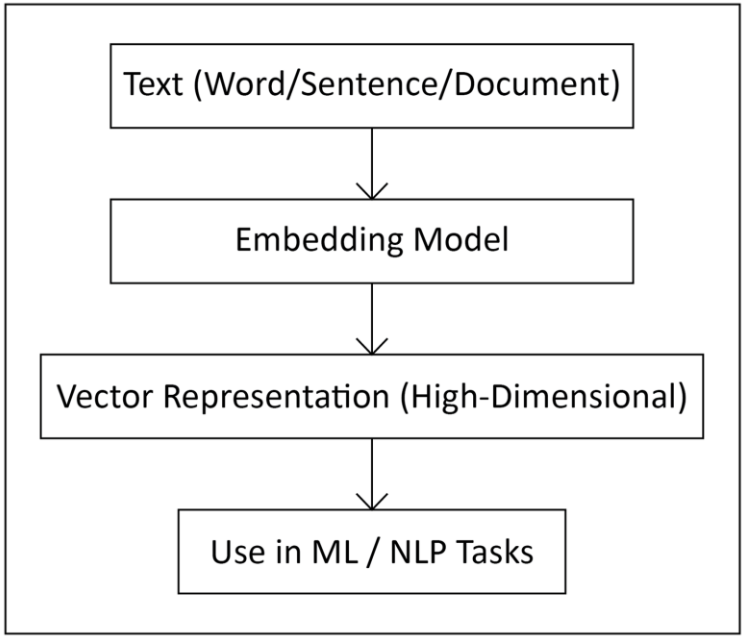


Male-Female



Verb Tense

Embedding Workflow



Why High Dimensions?

- **More dimensions** → better capture **subtle differences between objects**
- Each dimension can encode **different features or attributes**
- **Trade-off:** High-dimensional spaces → harder to visualize

*High-dimensional vector spaces allow more **precise similarity and clustering** of items.*

PCA for Embeddings

PCA (Principal Component Analysis) = a dimensionality reduction technique

- Projects **high-dimensional vectors** (e.g., embeddings) → **lower dimensions** (2D or 3D)
- Preserves **most of the variance** / important information in the data
- Helps **visualize patterns and clusters** in a simple space

Intuition

- High-dimensional embeddings contain rich semantic information
- PCA finds the **directions of maximum variance**
- By projecting onto top components, we can **see clusters and relationships** visually

*PCA is widely used to **inspect embeddings** and understand semantic grouping.*

CountVectorizer

(Transforming Text into Numerical Feature Vectors)

What is CountVectorizer?

CountVectorizer is a method that:

- Converts text into **numerical vectors**
- Based on **word occurrence counts**
- Produces a **Document–Term Matrix (DTM)**

It is a foundational technique in NLP for preparing text for machine learning.

How It Works

For a set of documents:

$$D = \{d_1, d_2, ..., d_n\}$$

Extract all unique words to form vocabulary:

$$V = \{t_1, t_2, ..., t_m\}$$

CountVectorizer creates matrix:

$$X \in \mathbb{R}^{n \times m}$$

Where every entry:

$$X_{ij} = \text{count of term } t_j \text{ in document } d_i$$

Tokenization Process

CountVectorizer performs:

1. Lowercasing
2. Removal of punctuation
3. Splitting text into words
4. Removing stopwords

Example: Text: **"The sky is blue, the sun is bright."**

Tokens: [the,sky,blue,the,sun,bright]

Vocabulary Creation

Example documents:

D1: “the sun rises in the east”

D2: “the sun sets in the west”

Vocabulary constructed:

V=[the,sun,rises,in,east,sets,west]

Vocabulary size = 7

Document–Term Matrix (DTM)

Document	the	sun	rises	in	east	sets	west
D1	2	1	1	1	1	0	0
D2	2	1	0	1	0	1	1

Matrix form:

$$X = \begin{bmatrix} 2 & 1 & 1 & 1 & 1 & 0 & 0 \\ 2 & 1 & 0 & 1 & 0 & 1 & 1 \end{bmatrix}$$

Vector Example

Sentence:

"the dog chased the cat"

Vocabulary:

[**the**, **dog**, **chased**, **cat**]

Vector Representation (Count Vector):

[**2**, **1**, **1**, **1**]

("the" appears twice, all other words appear once.)

Mathematical Properties

1. Sparsity

Most documents contain only a few words from the total vocabulary, so most matrix entries are zero.

$$\text{sparsity} = 1 - \frac{\text{number of non-zero elements}}{n \times m}$$

2. High Dimensionality

The vocabulary size can be very large, resulting in a **wide and high-dimensional Document–Term Matrix (DTM)**.

CountVectorizer Formula

For a term t_j in a document d_i, the entry X_{ij} in the Document–Term Matrix is defined as:

$$X_{ij} = \sum_{k=1}^{|d_i|} \mathbf{1}[\, d_i[k] = t_j \,]$$

- $|d_i|$ is the total number of words in the document di.
- $d_i[k]$ represents the k-th word in the document.
- The **indicator function** 1[·] works as:

$$\mathbf{1}[\text{condition}] = \begin{cases} 1, & \text{if the condition is true} \\ 0, & \text{otherwise} \end{cases}$$

- Essentially, we **scan each word in the document**. For every occurrence of t_j, we add 1. If the word is different, we add 0.
- Summing over all words gives the **total count of t_j in d_i**.

CountVectorizer Intuition

- Each row of the matrix represents a document.
- Each column represents a vocabulary term.
- Each entry tells you **how many times a specific term appears in that document**.

*This is the **mathematical foundation of Count Vectorizer**, which allows us to convert text into numerical features for machine learning.*

Advantages & Limitations of CountVectorizer

Advantages:

- Very simple and easy to implement
- Fast to compute, even for large datasets
- Interpretable: each feature corresponds to a word
- Scalable for large text corpora

Limitations:

- No semantic understanding of words
- Treats all words equally
- High dimensionality for large vocabularies
- Ignores word order and context

Term Frequency – Inverse Document Frequency

(A Mathematical Approach to Weighted Text Feature Extraction)

What is TF-IDF?

TF-IDF is a method that:

- Converts text into **numerical vectors**
- Assigns weights to words based on **importance**
- Reduces the impact of **common words**
- Produces a **weighted Document–Term Matrix (DTM)**

It is widely used in NLP for feature extraction

How It Works

For a set of documents:

$$D = \{d_1, d_2, ..., d_n\}$$

where d_i represents the i-th document in the corpus.

Vocabulary (unique terms in the corpus):

$$V = \{t_1, t_2, ..., t_m\}$$

where t_j represents the j-th term in the vocabulary, extracted from all documents.

TF-IDF Representation:

TF-IDF converts the raw text into a numerical Document–Term Matrix:

$$X \in \mathbb{R}^{n \times m}$$

where n = number of documents, m = number of terms in the vocabulary.

Each entry in the matrix is defined as:

$$X_{ij} = \text{tf-idf score of term } t_j \text{ in document } d_i$$

Matrix Representation

- **Rows:** Each row corresponds to a document.
- **Columns:** Each column corresponds to a term from the vocabulary.
- **Values:** Each entry X_{ij} represents the importance of term t_j in document d_i, adjusted for its frequency across the corpus.
- This matrix is then used as **numerical input** for machine learning or text analysis models.

Term Frequency (TF)

Term frequency measures how often a term appears in a document:

$$tf_{ij} = \frac{\text{count of term } t_j \text{ in document } d_i}{\text{total words in document } d_i}$$

Example:

Document = “the cat sat on the mat”

Term = “cat”

$$tf = \frac{1}{6} \approx 0.167$$

*(“cat” appears **once** in a document containing **six** total words.)*

Inverse Document Frequency (IDF)

IDF **reduces the weight of very common terms** and increases the weight of rare terms.

$$idf_j = \log\left(\frac{N}{df_j}\right)$$

Where:

- N = total number of documents
- df_j = number of documents that contain term t_j

Inverse Document Frequency (IDF)

Example

Term **“the”** appears in **5 out of 10 documents**:

$$idf = \log\left(\frac{10}{5}\right) = \log(2) \approx 0.301$$

TF-IDF

TF-IDF combines **Term Frequency (TF)** and **Inverse Document Frequency (IDF)** to measure how important a term is in a document relative to the entire corpus.

$$\text{tf-idf}_{ij} = tf_{ij} \times idf_j$$

Where:

- tf_{ij} = frequency of term t_j in document d_i
- idf_j = how rare the term t_j is across all documents

TF-IDF

TF component

- Measures *how often* a term appears in a specific document.
- Higher TF → term is more important within that document.

IDF component

- Measures *how rare* a term is across the entire corpus.
- Higher IDF → term appears in fewer documents → more meaningful.

Combined TF-IDF

- If a term appears **frequently in one document** but **rarely in others**, → it receives a **high TF-IDF score**.
- If a term appears **in most documents**, → its IDF is low → **low TF-IDF score**, even if its TF is high.
- If a term appears **rarely overall and rarely in the document**, → TF is low → final TF-IDF still remains low.

TF-IDF Example

Documents:

D1: “the sun rises in the east”
D2: “the sun sets in the west”

Vocabulary:

[the,sun,rises,in,east,sets,west]

Step1 : Compute**TF**

Step2 : Compute**IDF**

Step3 : Multiply TF × IDF → **TF-IDF matrix**

Mathematical Properties

Weighting Mechanism

- TF-IDF assigns **higher weights** to terms that are:
  - **Frequent within a document (high TF)**
  - **Rare across the entire corpus (high IDF)**
- Ensures meaningful terms dominate the feature space, not stopwords.
- Mathematically:
  - If a term appears in **all documents**,
    → IDF=log(N/N)=0
    → Weight becomes **0**.
  - If a term appears in **few documents**,
    → IDF is **large**, increasing influence in models.

Mathematical Properties

Sparsity of TF-IDF Matrix

- Vocabulary size grows with dataset → matrix becomes extremely large.
- But **each document contains only a small subset of all words**.
- Therefore, TF-IDF produces a **high-dimensional sparse matrix**, where most values are **0**.
- Useful for:
  - Memory-efficient sparse data structures
  - Fast linear models like SVM, Logistic Regression

Mathematical Properties

Normalization

- Documents naturally vary in length.
- Without normalization → long documents get unfairly high TF values.
- Common approach: **L2 normalization**

$$\vec{v}_{norm} = \frac{\vec{v}}{\sqrt{\sum_i v_i^2}}$$

- Ensures:
  - All document vectors have **unit length**
  - Better cosine similarity comparisons
  - Stable performance in ML algorithms (SVM, clustering)

Advantages & Limitations of TF-IDF

Advantages:

- Reduces weight of common words
- Highlights important and unique terms
- Useful for text classification and search
- Easy to interpret

Limitations:

- Does not capture word order or meaning
- Produces high-dimensional sparse vectors
- Requires preprocessing like tokenization and stopword removal

Word2Vec Embeddings

(Predictive Neural Representation of Words)

What Are Word2Vec Embeddings?

Word2Vec embeddings are:

- **Dense vector representations of words:** Each word → mapped to a continuous vector in $\mathbb{R}^d$ (e.g., 100–300 dimensions)
- **Generated using a shallow neural network**
  Two training methods:
  1. **CBOW (Continuous Bag of Words)**
  2. **Skip-Gram**
- **Capture semantic & syntactic relationships:** Words used in similar contexts will have similar vectors.
- **Preserve linear relationships in vector space**
  Example: **king – man + woman ≈ queen**

Why Word2Vec?

Traditional methods (their limitations):

Method	Weakness
CountVectorizer	Sparse vectors, no semantic meaning
TF–IDF	Still context-free, no relationships between words

Why Word2Vec is better:

Word2Vec **learns meaning** by training a small neural network to:

- **CBOW:** Predict the **target word** from surrounding **context words**
- **Skip-Gram:** Predict the **context words** from a **target word**

This forces the model to learn **semantic + syntactic patterns** in language, producing **dense, meaningful embeddings**.

Two Models in Word2Vec

1. CBOW (Continuous Bag of Words)

Goal: Predict the **target word** using its **surrounding context words**.

$$P(w_t \mid w_{t-2}, w_{t-1}, w_{t+1}, w_{t+2})$$

Properties:

- Fast to train
- Works well for **frequent words**
- Smooths noisy contexts

2. Skip-Gram

Goal: Predict the **context words** using the **target word**.

$$P(w_{t+k} \mid w_t)$$

(where k ∈ context window)

Properties:

- Better for **rare words**
- Learns richer semantic structure

Core Mathematical Idea (Word2Vec)

For a vocabulary of size **V** and embedding dimension **d**:

Each word has two embeddings

- **Input vector:** $v_w \in \mathbb{R}^d$
- **Output vector:** $v'_w \in \mathbb{R}^d$

These vectors are learned separately during training.

Skip-Gram Probability (Softmax)

The probability of predicting a **context word** w_o given the **center word** w_i:

$$P(w_o \mid w_i) = \frac{e^{v'^{T}_{w_o} v_{w_i}}}{\sum_{w=1}^{V} e^{v'^{T}_{w} v_{w_i}}}$$

- Numerator: Measures how similar the output embedding of w_o is to the input embedding of w_i.
- Denominator: Normalizes over all words using **softmax**, ensuring the output is a valid probability.

Objective Function (Skip-Gram)

The Skip-Gram model learns word embeddings by **maximizing the probability of predicting context words** given a center word.

Full Objective (Maximum Likelihood)

$$J = \sum_{(w_i, w_o) \in D} \log P(w_o \mid w_i)$$

Where:

- D = set of all (center word, context word) pairs in the corpus
- w_i = input/center word
- w_o = context/output word

Using the softmax probability: $P(w_o \mid w_i) = \frac{e^{{v'_{w_o}}^T v_{w_i}}}{\sum_{w=1}^{V} e^{{v'_w}^T v_{w_i}}}$

What the Optimization Learns

1. Semantic Directions

- Linear transformations encode meaning
- Example: $king - man + woman \approx queen$

2. Vector Similarity

Words with similar meaning → closer embeddings

- (car, vehicle)
- (doctor, physician)

3. Analogical Structure

$$Paris : France \approx Tokyo : Japan$$

Embeddings form **meaningful geometric relationships** in vector space.

Negative Sampling (Critical for Word2Vec)

Full softmax requires scoring **every word in the vocabulary (V)** → too slow for large corpora.

Negative Sampling replaces full softmax with a small classification task:

For each **true pair** (wi→wo):

- Keep **1 positive example**: the real context word w_o
- Sample **k negative words** $n_1, n_2, \ldots, n_k$ that are *not* context words

This reduces computation from **O(V)** to **O(k)**.

Negative Sampling Loss Function

$$\mathcal{L} = \log \sigma \left({v'_{w_o}}^T v_{w_i} \right) + \sum_{j=1}^{k} \log \sigma \left(-{v'_{n_j}}^T v_{w_i} \right)$$

Where:

- v_{w_i} = input embedding of center word
- v'_{w_o} = output embedding of the true context word
- v'_{n_j} = output embedding of negative (noise) words
- $\sigma(x) = \frac{1}{1+e^{-x}}$ (sigmoid)

Embeddings Learned

During training, Word2Vec learns **two vectors** for each word:

- Input vector → v_w
- Output vector → v'_w

1. Standard Word2Vec Embedding

Most implementations use only the **input vector**:

$$E(w) = v_w$$

This becomes the **final dense embedding** for word w.

2. Combined Embedding (Optional Variant)

Some versions improve semantic quality by **averaging** input and output vectors:

$$E(w) = \frac{v_w + v'_w}{2}$$

This mixes both representations:

- v_w captures how the word **predicts context**
- v'_w captures how the word is **predicted by context**

Example (Intuition)

Word2Vec arranges words in a vector space so that **semantic and syntactic relationships** are preserved.

Similar words lie close together

- “king” ↔ “queen”
- “doctor” ↔ “nurse”
- “walk” ↔ “walking”

Unrelated words are far apart

- “dog” far from “economy”

Famous Analogy

$$\text{king} - \text{man} + \text{woman} \approx \text{queen}$$

*This shows Word2Vec captures **directional meaning** (gender, tense, role, etc.) in vector space.*

Embedding Properties

Word2Vec embeddings naturally encode several important behaviors:

1. Similarity

Similarity between two word vectors A and B is measured using **cosine similarity**:

$$\cos(\theta) = \frac{A \cdot B}{\|A\| \, \|B\|}$$

Higher value → more similar meaning.

2. Analogy Reasoning

Word2Vec captures linear relationships:

- $\text{king} - \text{man} + \text{woman} \approx \text{queen}$
- $\text{Paris} - \text{France} + \text{Japan} \approx \text{Tokyo}$

3. Clustering Behavior

Words with similar meaning naturally **cluster together** in the vector space:

- Animals form a cluster
- Countries form a cluster
- Verbs form a cluster

Advantages & Limitations of Word2Vec

Advantages:

- Fast to train
- Captures **semantic similarity**
- Supports vector arithmetic (analogies)
- Outperforms Count/TF-IDF dramatically

Limitations:

- Context-free (same word → same embedding always)
- Cannot handle polysemy (“bank” issue)
- Requires large corpus for good performance
- Doesn’t use sentence-level context (unlike BERT)

BERT Embedding

(Contextual Semantic Representations using Transformers)

What Are BERT Embeddings?

BERT embeddings are **contextual vector representations** for words, sentences, or documents.

Key Characteristics

- **Context-aware:** The meaning of a word changes based on surrounding words (unlike TF–IDF or LSA).
- **Generated by Bidirectional Transformers:** BERT reads text **from left to right and right to left**, capturing deep semantic relationships.
- **Dense, fixed-size vectors:** Each word/sentence is converted into a compact numerical embedding (e.g., 768 dimensions).
- **Rich semantic meaning:** Similar words/sentences in meaning → embeddings are close in vector space.

Why BERT?

Limitations of Traditional Methods

Method	Weakness
Count Vectorizer	Only counts words — no meaning
TF–IDF	Still ignores context
LSA	Linear model — limited semantics

Why BERT is Better

- Provides **deep contextual understanding**
- Produces **different embeddings for the same word** depending on sentence meaning
  Example: “bank” (river bank) vs “bank” (finance)

BERT = Context + Semantics + Deep Representation

Core Mathematical Idea Behind BERT Embeddings

1. Token Embeddings

$$\mathbf{E}_{\text{token}}(w_i) \in \mathbb{R}^d$$

*Represents the **word itself** (similar to WordPiece vectors).*

2. Position Embeddings

$$\mathbf{E}_{\text{position}}(i) \in \mathbb{R}^d$$

*Adds information about the **token's position** in the sentence (since Transformers have no inherent order).*

3. Segment Embeddings

$$\mathbf{E}_{\text{segment}}(s) \in \mathbb{R}^d$$

*Indicates **which sentence** a token belongs to (Sentence A or B in next-sentence tasks).*

Core Mathematical Idea Behind BERT Embeddings

Final Input Embedding

For each token w_i:

$$\boxed{\mathbf{X}_i = \mathbf{E}_{\text{token}}(w_i) + \mathbf{E}_{\text{position}}(i) + \mathbf{E}_{\text{segment}}(s)}$$

This combined vector is fed into the **Transformer layers**, enabling BERT to model deep contextual meaning.

For a full input sequence of length L:

$$\mathbf{X} = \begin{bmatrix} \mathbf{X}_1 \\ \mathbf{X}_2 \\ \vdots \\ \mathbf{X}_L \end{bmatrix} = \begin{bmatrix} \mathbf{E}_{\text{token}}(w_1) + \mathbf{E}_{\text{position}}(1) + \mathbf{E}_{\text{segment}}(s_1) \\ \mathbf{E}_{\text{token}}(w_2) + \mathbf{E}_{\text{position}}(2) + \mathbf{E}_{\text{segment}}(s_2) \\ \vdots \\ \mathbf{E}_{\text{token}}(w_L) + \mathbf{E}_{\text{position}}(L) + \mathbf{E}_{\text{segment}}(s_L) \end{bmatrix}$$

Transformer Encoder Layers (BERT)

BERT uses **L identical Transformer encoder layers**. Each layer has **two main components**:

Multi-Head Self-Attention

For a single attention head, the attention function is:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V$$

Where:

- $Q = XW_Q$ (queries)
- $K = XW_K$ (keys)
- $V = XW_V$ (values)
- d_k = dimension of keys
- Softmax is applied row-wise

Multi-Head Version

$$\text{MultiHead}(Q, K, V) = \text{Concat}(h_1, h_2, \ldots, h_H)W_O$$

Where each head:

$$h_j = \text{Attention}(QW_Q^{(j)}, KW_K^{(j)}, VW_V^{(j)})$$

Feed-Forward Network (FFN)

Each token vector passes through a **2-layer MLP** with a ReLU:

$$\text{FFN}(x) = W_2\,\text{ReLU}(W_1 x + b_1) + b_2$$

Where:

- $W_1 \in \mathbb{R}^{d_{\text{model}} \times d_{\text{ff}}}$
- $W_2 \in \mathbb{R}^{d_{\text{ff}} \times d_{\text{model}}}$
- $d_{\text{ff}} \approx 4 \times d_{\text{model}}$

Final Output

After stacking all **L layers**:

$$\text{Contextual Embeddings} = \text{Encoder}_L(\text{Input Embeddings})$$

These embeddings contain **deep contextual meaning** for every token.

Bidirectional Attention

BERT learns context from both left and right.

BERT does **not** read text only left→right or right→left. It reads the **entire sentence at once**.

Contextual Embedding

$$\text{Embedding}(w_i) = f(w_1, w_2, \ldots, w_n)$$

The embedding of word $\mathbf{w_i}$ depends on **all** words in the sentence — before and after it.

Why this matters: The meaning of a word changes with its neighbors.

Example

- "river **bank**" → *bank* = *riverside*
- "open an account at the **bank**" → *bank* = *financial institution*

BERT gives completely different embeddings for the same word based on context.

Types of BERT Embeddings

1. Token Embeddings (per word-piece)

Each token (WordPiece) gets a 768-dimensional embedding in BERT-Base.

$$\text{TokenEmb}(w_i) \in \mathbb{R}^{768}$$

2. CLS Sentence Embedding

The **[CLS]** token's final hidden state represents the **entire sentence**.

$$\text{SentenceEmbedding} = h_{[\text{CLS}]}$$

3. Mean Pooling Sentence Embedding

Average of all token embeddings:

$$E_{\text{sent}} = \frac{1}{n} \sum_{i=1}^{n} h_i$$

Where:

- n = number of tokens
- h_i = final hidden state of token i

Embedding Dimensions

Typical BERT Model Sizes

BERT Model	Embedding Dimension	# of Layers (L)	# of Attention Heads
BERT-Base	768	12	12
BERT-Large	1024	24	16

- More layers → deeper semantics
- More heads → richer attention patterns
- Larger dimensions → more expressive embeddings

Mathematical Representation of BERT Output

1. Final Contextual Embedding of Token w_i

Every token embedding after all Transformer encoder layers:

$$h_i = \text{TransformerEncoder}(X_1, X_2, \ldots, X_n)$$

Where:

- X_i = token + position + segment embedding
- $h_i \in \mathbb{R}^d$ (d = 768 for Base, 1024 for Large)

2. Sentence Embedding Using [CLS]

$$E_{\text{sentence}} = h_{[\text{CLS}]}$$

The [CLS] token's final hidden state represents the whole sentence.

3. Sentence Embedding Using Average Pooling

$$E_{\text{sentence}} = \frac{1}{n} \sum_{i=1}^{n} h_i$$

Average of all token embeddings.

Applications of BERT Embeddings

BERT embeddings are widely used in modern NLP tasks:

- **Text Classification** – sentiment analysis, spam detection
- **Semantic Search** – finding contextually relevant documents
- **Document Clustering** – grouping similar texts
- **Question Answering** – extractive QA systems
- **Summarization** – generating concise summaries
- **Similarity Analysis** – computing semantic similarity between texts

Advantages & Limitations of BERT Embedding

Advantages:

- Captures deep semantics
- Contextual meaning
- Handles ambiguous words
- State-of-the-art performance in NLP
- Dense & continuous embedding space

Limitations:

- Heavy computation
- Requires GPU for large datasets
- Long sequence limit (512 tokens)
- No explicit knowledge beyond training data

BM25

(Probabilistic Ranking Function)

What is BM25?

BM25 (Best Match 25) is a **probabilistic ranking function** used in search engines to score how relevant a document is to a user's query.

Key Points

- Developed from the **probabilistic retrieval framework**
- Improves on TF–IDF by adding **term saturation** and **document length normalization**
- Used in modern search systems like **Elasticsearch, Lucene, Whoosh**

Purpose

- Assigns a **relevance score** for each document
- Higher score = more likely the document matches the query

Intuition Behind BM25

M25 scores a document higher when it:

1. Contains the query terms:

- More occurrences of the query words → higher relevance.

2. Uses those terms with meaningful frequency (TF):

- But **not linearly** — BM25 applies *diminishing returns*:
  - More repetitions help, but after a point, extra occurrences add little benefit.

3. Is not excessively long:

- Long documents naturally contain more words → BM25 applies a **length normalization penalty** to avoid bias.

4. Uses rare terms (IDF weighting): Words that appear in few documents are more informative, so BM25 rewards them.

BM25 Formula

For a document **D** and query **Q**:

$$\text{BM25}(D, Q) = \sum_{t \in Q} \text{IDF}(t) \times \frac{tf(t, D)\,(k_1 + 1)}{tf(t, D) + k_1 \left(1 - b + b \cdot \frac{|D|}{\text{avgDL}}\right)}$$

Where:

- tf(t,D) = term frequency of term *t*
- |D| = length of document
- avgDL = average document length in corpus
- k_1 = term frequency saturation (1.2–2.0)
- b = length normalization factor (0.75)

IDF in BM25

The **Inverse Document Frequency (IDF)** used in BM25 is:

$$\mathrm{IDF}(t) = \ln\left(\frac{N - df(t) + 0.5}{df(t) + 0.5} + 1\right)$$

Where:

- **N** = total number of documents
- **df(t)** = number of documents that contain term **t**

Rare terms *(low df(t))* → ***higher IDF***

Common terms *(high df(t))* → ***lower IDF***

Rare terms get ***more weight*** *in BM25, improving relevance scoring.*

Clustering

(Grouping Documents or Words into Meaningful Clusters)

What is Clustering?

Clustering = Grouping similar items based on their characteristics

- Items within the same cluster are highly similar
- Items in different clusters are dissimilar
- No labels are provided — it is completely unsupervised

Why Use Embeddings for Clustering?

Traditional text features (CountVectorizer, TF–IDF):

- rely only on **keywords**
- ignore **meaning** and **context**

Embeddings (Word2Vec / BERT / Sentence Transformers):

- Capture semantic meaning
- Represent sentences in **high-dimensional vector space**
- Semantically similar sentences → **closer vectors**
- Dissimilar sentences → **far apart**

Result: More accurate and meaningful clustering

Mathematical Intuition

1. **Sentence → Vector**

$$v = \text{Embedding}(\text{sentence})$$

Each sentence is mapped to a **high-dimensional vector**.

2. **Clustering Objective**

Group vectors by **similarity** (or minimize distance):

- **Euclidean distance:** $\text{dist}(v_i, v_j) = \|v_i - v_j\|_2$
- **Cosine similarity:** $\text{cosine\_sim}(v_i, v_j) = \frac{v_i \cdot v_j}{\|v_i\|\|v_j\|}$

3. **Interpretation**

- **High cosine similarity** → sentences are semantically similar
- **Small Euclidean distance** → sentences are close in vector space

*Clustering algorithms group **close vectors** together to form meaningful clusters.*

K-Means for Embedding Clustering

Objective: Minimize the sum of squared distances between points and their cluster centroids:

$$\min \sum_{i=1}^{k} \sum_{x \in C_i} ||x - \mu_i||^2$$

Where:

- x = embedding vector of a sentence or word
- μ_i = centroid of cluster C_i
- k = number of clusters

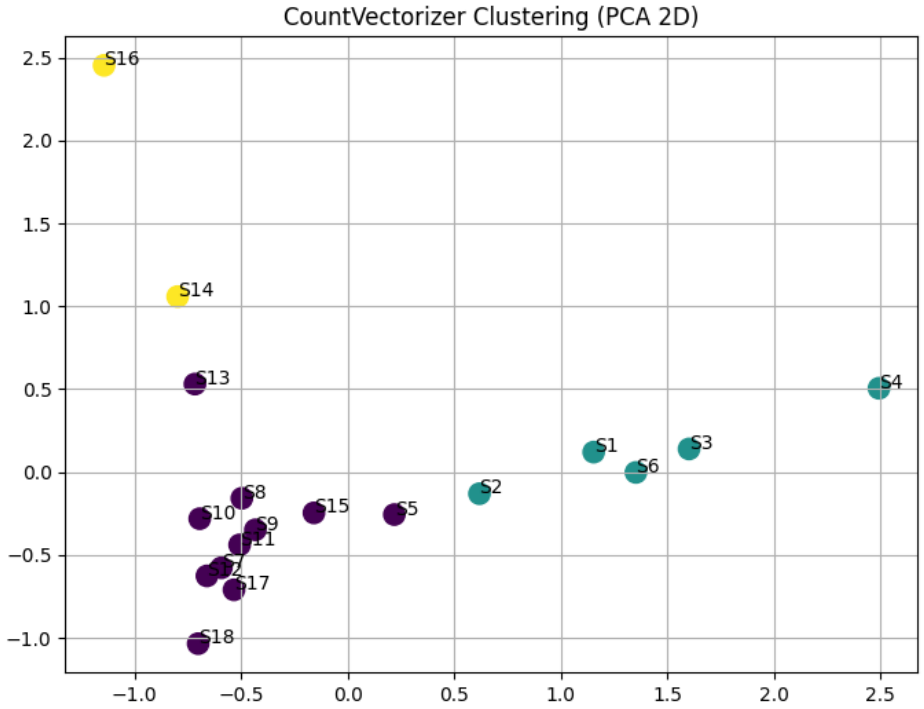
Intuition:

- Clusters are formed by grouping points **closest to their centroid**
- Embedding vectors that are semantically similar → grouped together

K-Means efficiently organizes high-dimensional embeddings into meaningful clusters.

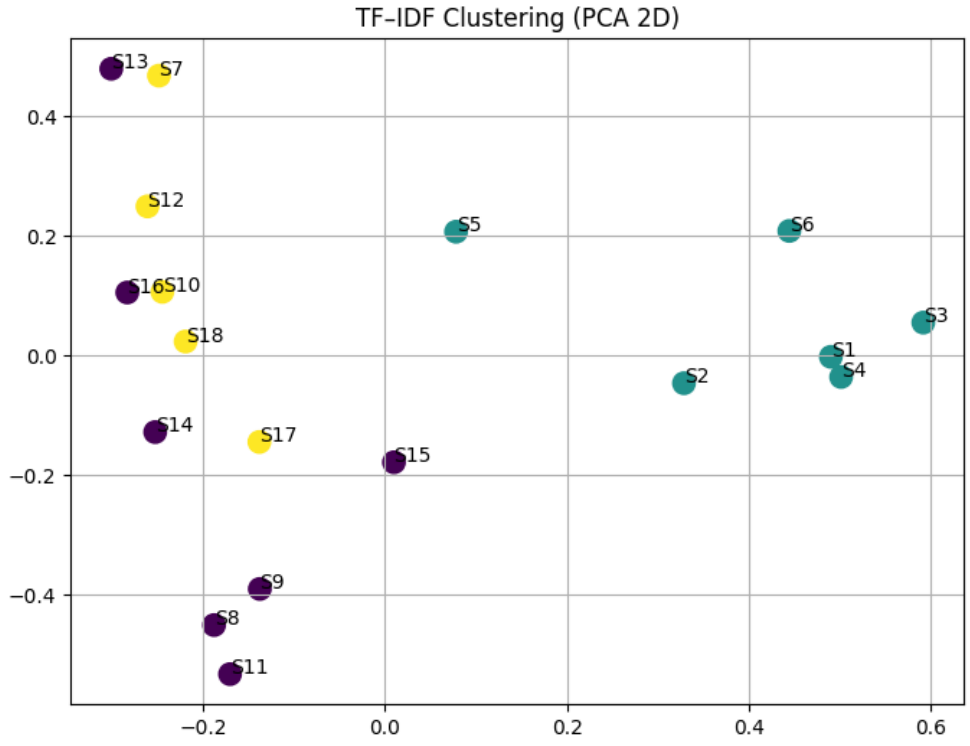
CountVectorizer

```python
sentences = [
    "Delhi is the capital of India.",
    "Tokyo is a major city in Japan.",
    "Paris is known as the city of lights.",
    "New York is one of the busiest cities in the world.",
    "London is famous for its historical landmarks.",
    "Sydney is a coastal city known for the Opera House.",
    "LLMs are used for text processing.",
    "Transformers help machines understand language.",
    "NLP models learn linguistic patterns.",
    "Tokenization converts text into numerical units.",
    "Language models generate human-like responses.",
    "Text classification assigns labels to sentences.",
    "CNNs are popular for image classification.",
    "Vision Transformers perform well on image tasks.",
    "Object detection identifies items in pictures.",
    "Image segmentation divides an image into meaningful regions.",
    "Face recognition systems detect and identify faces.",
    "Optical character recognition extracts text from images."
]
```



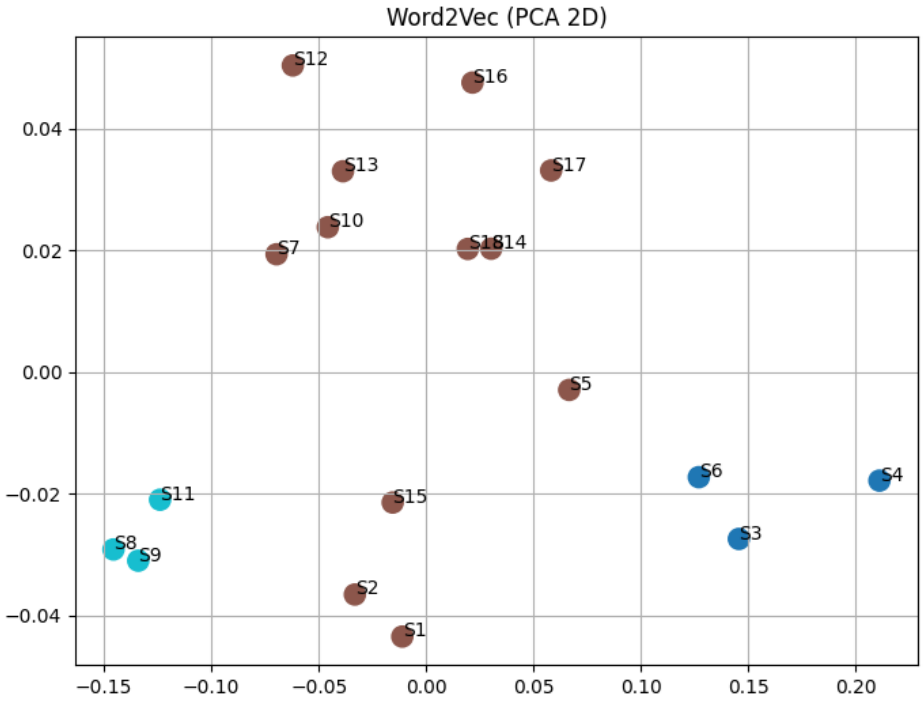
TF-IDF Embedding

```python
sentences = [
    "Delhi is the capital of India.",
    "Tokyo is a major city in Japan.",
    "Paris is known as the city of lights.",
    "New York is one of the busiest cities in the world.",
    "London is famous for its historical landmarks.",
    "Sydney is a coastal city known for the Opera House.",
    "LLMs are used for text processing.",
    "Transformers help machines understand language.",
    "NLP models learn linguistic patterns.",
    "Tokenization converts text into numerical units.",
    "Language models generate human-like responses.",
    "Text classification assigns labels to sentences.",
    "CNNs are popular for image classification.",
    "Vision Transformers perform well on image tasks.",
    "Object detection identifies items in pictures.",
    "Image segmentation divides an image into meaningful regions.",
    "Face recognition systems detect and identify faces.",
    "Optical character recognition extracts text from images."
]
```



Word2Vec Embedding

```python
sentences = [
    "Delhi is the capital of India.",
    "Tokyo is a major city in Japan.",
    "Paris is known as the city of lights.",
    "New York is one of the busiest cities in the world.",
    "London is famous for its historical landmarks.",
    "Sydney is a coastal city known for the Opera House.",
    "LLMs are used for text processing.",
    "Transformers help machines understand language.",
    "NLP models learn linguistic patterns.",
    "Tokenization converts text into numerical units.",
    "Language models generate human-like responses.",
    "Text classification assigns labels to sentences.",
    "CNNs are popular for image classification.",
    "Vision Transformers perform well on image tasks.",
    "Object detection identifies items in pictures.",
    "Image segmentation divides an image into meaningful regions.",
    "Face recognition systems detect and identify faces.",
    "Optical character recognition extracts text from images."
]
```



BERT Embedding

```python
sentences = [
    "Delhi is the capital of India.",
    "Tokyo is a major city in Japan.",
    "Paris is known as the city of lights.",
    "New York is one of the busiest cities in the world.",
    "London is famous for its historical landmarks.",
    "Sydney is a coastal city known for the Opera House.",
    "LLMs are used for text processing.",
    "Transformers help machines understand language.",
    "NLP models learn linguistic patterns.",
    "Tokenization converts text into numerical units.",
    "Language models generate human-like responses.",
    "Text classification assigns labels to sentences.",
    "CNNs are popular for image classification.",
    "Vision Transformers perform well on image tasks.",
    "Object detection identifies items in pictures.",
    "Image segmentation divides an image into meaningful regions.",
    "Face recognition systems detect and identify faces.",
    "Optical character recognition extracts text from images."
]
```